

پروژه سیستم هوشمند اعلام و اطفاء حریق

1. مقدمه

در محیط‌های صنعتی و خانگی مدرن، زمان واکنش به حوادثی نظیر آتش‌سوزی یا نشت گاز، مرز بین کنترل حادثه و بروز یک فاجعه را تعیین می‌کند. تکیه بر نیروی انسانی برای مانیتورینگ مداوم، درصد خطای بالایی دارد و در شرایط بحرانی، سرعت عمل انسان به شدت کاهش می‌یابد. هدف از این پروژه، طراحی یک سیستم تعبیه‌شده (Embedded System) کاملاً خودکار، بلادرنگ (Real-Time) و امنیتی مبتنی بر میکروکنترلر STM32 است. این سیستم وظیفه دارد به صورت مداوم متغیرهای محیطی را پایش کرده و در صورت بروز خطر، بدون نیاز به دخالت انسان، واکنش‌های فیزیکی سریع (نظیر قطع شیر اصلی گاز یا فعال‌سازی پمپ آب) را انجام دهد. همچنین، یک لایه امنیتی یکپارچه (رابط کاربری با کیپد و LCD) برای جلوگیری از دستکاری غیرمجاز سیستم توسط افراد متفرقه طراحی شده است.

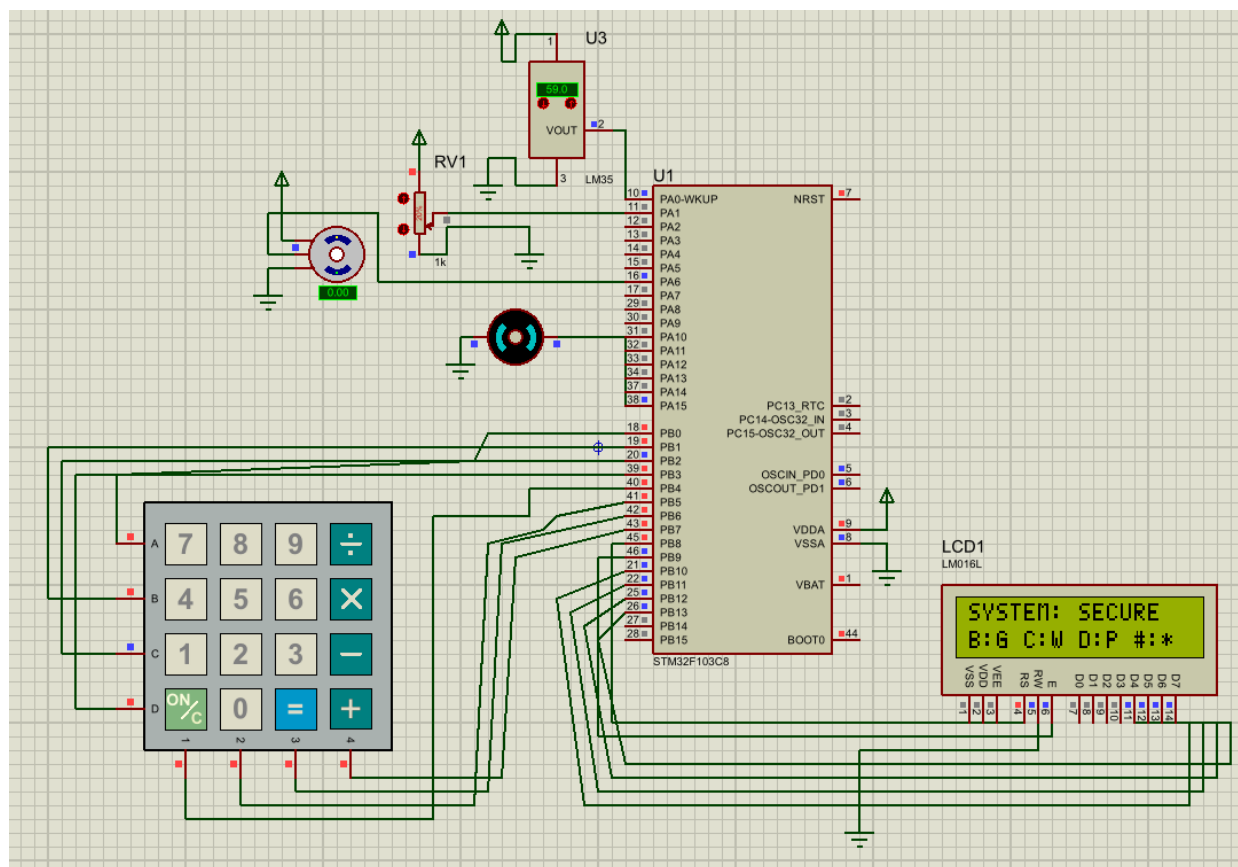
2. بیان مسأله

سیستم‌های هشداردهنده سنتی و پروژه‌های ساده دانشجویی عموماً از ساختار سرکشی مداوم (Polling) و تاخیرهای مسدودکننده (نظیر دستور Delay) استفاده می‌کنند. این معماری یک نقطه ضعف اساسی دارد: «خفگی پردازنده». زمانی که میکروکنترلر در حال انتظار برای فشردن یک کلید یا چاپ یک متن روی نمایشگر است، کاملاً نسبت به سنسورهای حیاتی کور می‌شود.

علاوه بر این، در سیستم‌های ساده، اگر دو بحران (مثل حریق و نشت گاز) همزمان رخ دهند، تداخل منطقی ایجاد می‌شود. مسأله اصلی در این پروژه، طراحی معماری پیشرفته‌ای بود که بتواند بدون از دست دادن حتی یک میلی‌ثانیه، همزمان کیبورد را بخواند، سنسورها را چک کند، وضعیت موتورها را تغییر دهد و رمز عبور را پردازش کند.

3. معماری سخت‌افزار و شبیه‌سازی در پروتئوس

برای پیاده‌سازی این سیستم، از نرم‌افزار **Proteus** جهت شبیه‌سازی فیزیکی و از میکروکنترلر **STM32F103C8** به عنوان واحد پردازش مرکزی استفاده شد.



شکل 3.1: شماتیک کامل پروتئوس

۱. شبیه‌سازی سنسورها و کالیبراسیون

- سنسور حرارت (LM35): این سنسور به ازای هر درجه سانتی گراد، 10mV خروجی آنالوگ تولید می‌کند. با توجه به اینکه مدل LM35 در پروتئوس با مرجع 5V کار می‌کند، برای فعال‌سازی آلارم در دمای استاندارد 60 درجه سانتی گراد (0.6 ولت)، مقدار آستانه (Threshold) مبدل 12 بیتی (ADC) با فرمول دقیق کالیبره و روی عدد 491 تنظیم شد.
- سنسور گاز (شبیه‌سازی با پتانسیومتر): در دنیای واقعی، سنسورهای گاز (مانند سری MQ) بر اساس غلظت گاز، ولتاژ آنالوگ متغیری تولید می‌کنند. در این شبیه‌سازی، از یک پتانسیومتر متصل به پایه PA1

استفاده شد تا به صورت دقیق افت و خیز ولتاژ یک سنسور گاز واقعی را شبیه‌سازی کند. آستانه خطر نشت گاز روی 20% مقیاس کامل ADC (عدد 819) تنظیم گردید.

II. عملگرهای مکانیکی (Actuators)

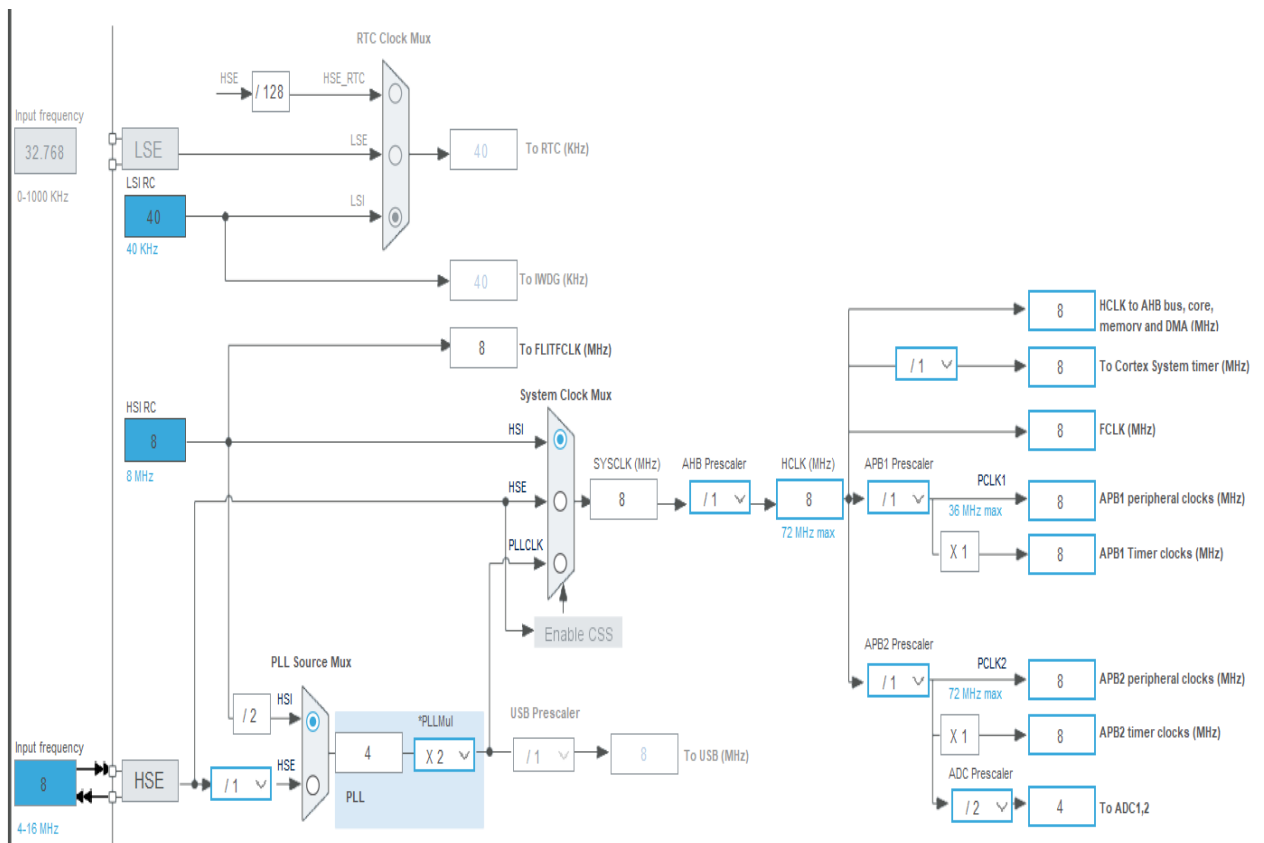
- پمپ آب (DC Motor): به پایه PA15 متصل شده و توسط پالس PWM تایمر ۲ با حداکثر توان (Duty Cycle = 999) در زمان حریق روشن می‌شود.
- شیر برقی گاز (Servo Motor): در صنعت، باز و بسته کردن ناگهانی شیرهای گاز خطرناک است و به موتورهای کنترل موقعیت نیاز است. به همین دلیل از سروو موتور متصل به PA6 (تایمر ۳) استفاده کردیم. با کالیبراسیون دقیق دیوتی سایکل در فرکانس 50Hz، زاویه موتور در حالت ایمن روی 0.0 درجه (پالس ۱ میلی‌ثانیه معادل مقدار رجیستر 100) و در حالت خطر دقیقاً روی 90.0 درجه (پالس ۲ میلی‌ثانیه معادل مقدار رجیستر 200) قفل می‌شود تا شیر فیزیکی کاملاً مسدود گردد.

4. ایده‌هایی که پیاده‌سازی کردیم (معماری و سخت‌افزار)

برای غلبه بر چالش‌های بیان‌شده، به جای کدنویسی خطی، از معماری «ماشین حالت (State Machine)» و «وقفه‌های سخت‌افزاری (Hardware Interrupts)» استفاده کردیم.

I. تنظیمات کلاک و زمان‌بندی (Clock Tree)

مغز متفکر سیستم، میکروکنترلر STM32F103C8 است. ما کلاک سیستم را روی نوسان‌ساز داخلی (HSI) با فرکانس دقیق ۸ مگاهرتز (8 MHz) تنظیم کردیم. این فرکانس پایه، مرجع تمام محاسبات زمانی، تایمرها و وقفه‌های ما در کل پروژه است.

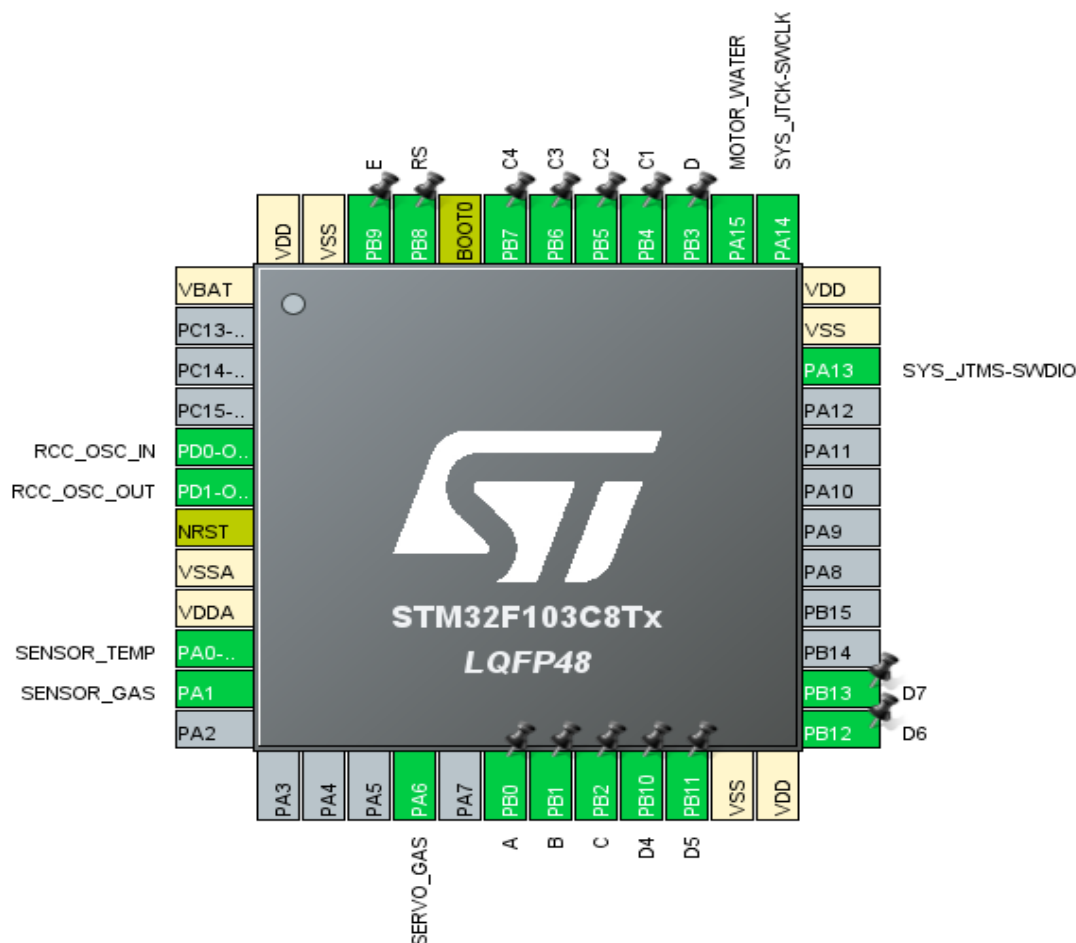


شکل 3.1: تنظیمات Clock Tree در نرم افزار CubeMX

II. معماری پین ها (Pinout Configuration)

سیستم به صورت ماژولار طراحی شده تا تداخلی بین بخش های آنالوگ و دیجیتال پیش نیاید:

- سنسور دما (LM35 – پایه PA0): متصل به کانال ۰ مبدل آنالوگ به دیجیتال (ADC1).
- سنسور گاز (پتانسیومتر شبیه ساز – پایه PA1): متصل به کانال ۱ مبدل ADC1.
- پورت نمایشگر (LCD 16x2): پایه های PB8 تا PB13 به صورت خروجی دیجیتال برای ارسال دیتا به LCD پیکربندی شدند.
- کیپد ماتریسی (x44): برای جلوگیری از نویز، ستون ها به پایه های ورودی PB4 تا PB7 (با مقاومت Pull-Up داخلی) و سطرها به خروجی های PB0 تا PB3 متصل شدند.



شکل 3.2: Pinout میکروکنترلر در نرم افزار CubeMX

III. کنترل عملگرها با سیگنال PWM (تایمرهای ۲ و ۳)

برای کنترل موتورها بدون درگیر کردن پردازنده اصلی، از تایمرهای سخت افزاری استفاده کردیم:

- شیر گاز (Servo Motor – پایه PA6): سروو موتورها برای کارکرد دقیق به فرکانس ۵۰ هرتز (دوره تناوب ۲۰ میلی ثانیه) نیاز دارند. ما تایمر ۳ را با فرکانس پایه ۸ مگاهرتز تغذیه کردیم. با تنظیم Prescaler روی 79، کلاک تایمر به ۱۰۰ کیلوهرتز کاهش یافت و با تنظیم ARR روی 1999، دقیقاً به فرکانس ۵۰ هرتز رسیدیم. حالا با ارسال پالس ۱۰۰، شیر باز (زاویه ۰) و با پالس ۲۰۰، شیر بسته (زاویه ۹۰ درجه) می شود.
- پمپ آب (DC Motor – پایه PA15): با استفاده از کانال ۱ تایمر ۲، سرعت پمپ آب کنترل می شود که در زمان حریق، با حداکثر توان (پالس ۹۹۹) فعال می گردد.

IV. اسکن غیرمسدودکننده کیپد (Non-Blocking Keypad)

چرا کیپد با وقفه خارجی (EXTI) پیاده‌سازی نشد؟

یک اشتباه رایج، اتصال پایه‌های کیپد به وقفه خارجی (External Interrupt) است. کلیدهای مکانیکی دارای پدیده «لرزش کنتاکت» یا Bounce هستند. اگر از EXTI استفاده می‌کردیم، فشردن یک کلید باعث ده‌ها وقفه پشت‌سرهم شده و پردازنده را فلج می‌کرد.

راهکار ما: اسکن غیرمستدودکننده با تایمر ۴ (TIM4)

ما تایمر ۴ (TIM4) را طوری تنظیم کردیم که هر ۱۰ میلی‌ثانیه یک وقفه (Interrupt) ایجاد کند. پردازنده هر ۱۰ میلی‌ثانیه کار اصلی خود را متوقف کرده، در حد چند میکروثانیه فقط یک سطر از کیپد را چک می‌کند و سریعاً به خواندن سنسورها برمی‌گردد. برای حذف لرزش کلید، سیستم شمارنده‌ای طراحی شد که باید ۱۵۰ میلی‌ثانیه (۱۵ وقفه متوالی) عدم فشردن کلید را رصد کند تا اجازه دریافت کلید جدید داده شود. به این ترتیب سیستم کاملاً بلادرنگ (Real-Time) ماند.

۷. عملکرد کلیدهای کیپد و بازخورد منوها روی LCD (مسیریابی وابسته به حالت)

کیپد ماتریسی ۴×۴ در این پروژه به عنوان رابط اصلی انسان و ماشین (HMI) عمل می‌کند. برای بالا بردن ضریب اطمینان و جلوگیری از خطای اپراتور، منطق نرم‌افزاری این بخش به صورت مسیریابی وابسته به حالت

(State-Dependent Routing) طراحی شده است. این یعنی یک کلید مشخص (مثل B یا C)، بر اساس وضعیت فعلی موتورها، منوی متفاوتی را روی LCD فراخوانی می‌کند:

- **کلید A (لغو عمومی و بازگشت):** در هر منویی (به جز وضعیت آلارم‌های اصلی) اگر کاربر کلید A را فشار دهد، سیستم بلافاصله حافظه موقت ورودی را با دستور memset پاک کرده، عملیات را لغو می‌کند و با نمایش پیام CANCELED... به صفحه اصلی (SYSTEM: SECURE) برمی‌گردد.
- **کلید B (منوی دوگانه شیر گاز):** عملکرد این کلید به وضعیت فیزیکی سروو موتور بستگی دارد:
 - اگر شیر گاز باز باشد، با فشردن B، منوی SHUT GAS [A=Cncl] (بستن گاز) ظاهر می‌شود.
 - اگر شیر گاز بسته باشد، با فشردن B، منوی OPEN GAS [A=Cncl] (باز کردن گاز) ظاهر می‌شود.پس از وارد کردن رمز ۴ رقمی، کاراکترها روی LCD به صورت *ماسک می‌شوند تا امنیت حفظ شود. در صورت تایید، LCD پیام صریح GAS VLV: OPEN یا GAS VLV: CLOSED را نمایش داده و خروجی را اعمال می‌کند.
- **کلید C (منوی دوگانه پمپ آب):** این کلید نیز به صورت پویا وضعیت پمپ آب (DC Motor) را چک می‌کند:

- اگر پمپ خاموش باشد، منوی **STRT WTR [A=Cncl]** (روشن کردن آب پاش) فعال می شود.
- اگر پمپ روشن باشد، منوی **STOP WTR [A=Cncl]** (خاموش کردن آب پاش) فعال می شود. پس از ورود رمز موفق، بازخورد مستقیم به صورت **WATER PUMP: ON** یا **WATER PUMP: OFF** روی نمایشگر چاپ می شود.
- کلید **D** (تغییر رمز دو مرحله ای): ابتدا رمز فعلی را می پرسد، در صورت تایید، امکان درج رمز جدید را فراهم کرده و آن را در حافظه ذخیره می کند.
- **کلیدهای نمایش زنده (*, #):** با فشردن کلید * دمای زنده محیط (به درجه سانتی گراد) و با کلید # غلظت گاز (به درصد) با تبدیل فوری روی LCD نمایش داده می شود.

VI. منطق اولویت بحران (Priority Logic)

سیستم دارای یک ساختار اولویت بندی صنعتی است:

1. **اولویت اول (حریق):** در صورت تشخیص دمای بالای ۶۰ درجه، سیستم هم پمپ آب را روشن می کند و هم برای جلوگیری از انفجار، شیر گاز را می بندد.
2. **اولویت دوم (گاز):** در صورت نشت گاز (عبور از مرز 20٪)، فقط شیر گاز بسته می شود. اگر حین نشت گاز، حریق هم رخ دهد، سیستم بلافاصله به اولویت اول ارتقا می یابد.

[تصویر از شبیه سازی پروتئوس در حالت عادی (SYSTEM SECURE) را اینجا قرار دهید]

[تصویر از شبیه سازی پروتئوس در حالت حریق (FIRE ALARM) با روشن بودن موتورها را اینجا قرار دهید]

VII. معماری بلادرنگ Bare-Metal و سیستم وقفه های سخت افزاری

یک تصور اشتباه مهندسی این است که برای داشتن یک سیستم بلادرنگ (Real-Time)، حتماً باید یک سیستم عامل مثل FreeRTOS روی میکروکنترلر نصب باشد. این پروژه دقیقاً اثبات می کند که چطور می توان بدون سیستم عامل و به صورت Bare-Metal (کدنویسی مستقیم روی سخت افزار) یک سیستم بلادرنگ قطعی (Deterministic) ساخت.

- **مفهوم وقفه (Interrupt)** در این پروژه: در کدهای معمولی، میکروکنترلر در یک حلقه دائم بچرخد و دکمه ها را چک کند این کار پردازنده را فلج می کند. ما در این پروژه از واحد **NVIC** (کنترل کننده وقفه های میکروکنترلر) استفاده کردیم. پردازنده ما ۱۰۰٪ وقت خود را در حلقه اصلی (**while**) صرف خواندن

سنسورهای آنالوگ دما و گاز می‌کند. اما به محض اینکه تایمر ۴ (TIM4) علامت بدهد، پردازنده در یک میکروثانیه کار خود را رها کرده، به بخش کپی می‌رود، آن را بررسی می‌کند و فوراً به حلقه اصلی برمی‌گردد.

- چرا سیستم ما بلادرنگ (Real-Time) است؟ تعریف سیستم بلادرنگ این است که سرعت پاسخگویی سیستم به رویدادهای خارجی (مثل آتش‌سوزی) سقف زمانی مشخصی داشته باشد و معطل کدهای دیگر نماند. در کد ما، چون اسکن کپی و نمایشگر در پس‌زمینه و توسط وقفه‌های سخت‌افزاری تایمر مدیریت می‌شوند، حلقه اصلی کد هرگز معطل فشرده شدن یک دکمه یا تایپ شدن یک متن روی LCD نمی‌ماند. به همین دلیل، سنسور دما و گاز در هر ثانیه هزاران بار بدون کوچک‌ترین وقفه یا لگ نرم‌افزاری خوانده می‌شوند. این یعنی واکنش سیستم به خطر حریق کاملاً آنی و بلادرنگ است، بدون اینکه نیاز به سیستم عامل سنگینی مثل FreeRTOS داشته باشیم.

VIII. زمان‌بندی دقیق سخت‌افزاری و همگام‌سازی حافظه

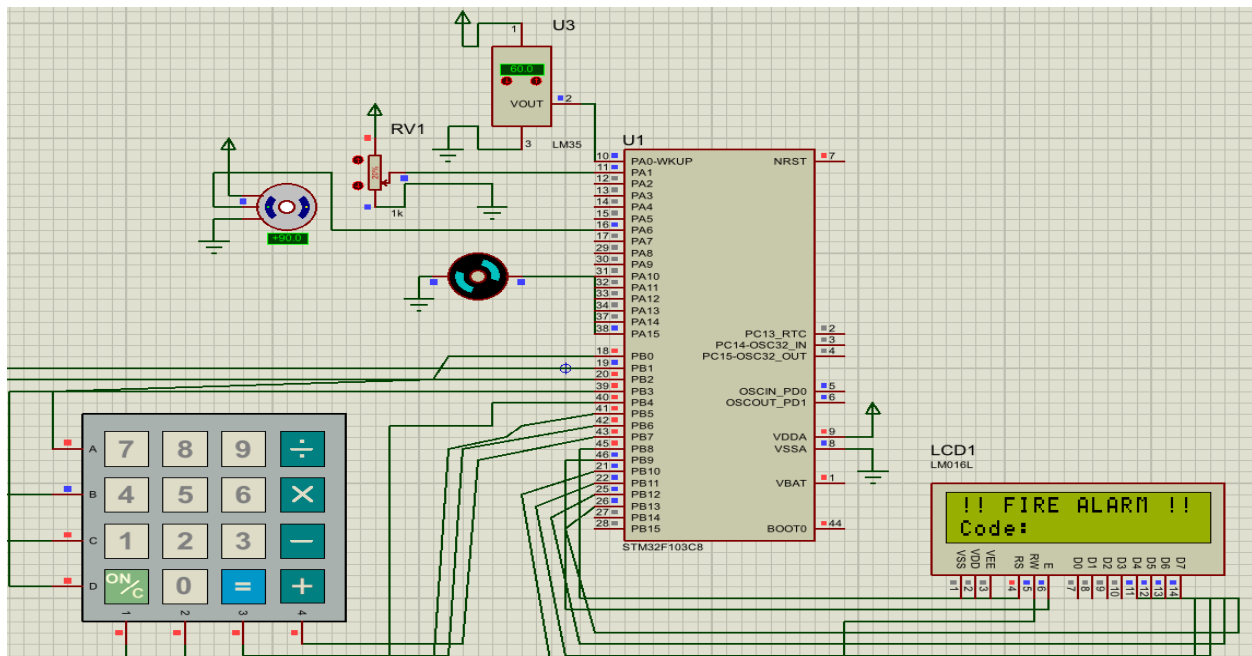
یکی از بزرگ‌ترین بخش‌های مهندسی این پروژه، هماهنگ کردن سرعت مانتورینگ میکروکنترلر (که در حد میکروثانیه است) با سرعت قطعات مکانیکی و فیزیکی شبیه‌ساز پروتئوس بود. برای این کار سه تکنیک زمان‌بندی اعمال شد:

- چرخه اسکن ۱۰ میلی‌ثانیه‌ای سطرها: در الگوریتم وقفه تایمر ۴، ما در هر بازه ۱۰ میلی‌ثانیه‌ای فقط و فقط یک سطر از کپی را LOW (فعال) می‌کنیم و پردازنده بین رها می‌کند تا وقفه بعدی. این تاخیر ۱۰ میلی‌ثانیه‌ای تعمدی، به موتور فیزیکی پروتئوس و کلید مکانیکی فرصت می‌دهد تا افت ولتاژ روی سیم‌ها را محاسبه کند و مانع از سوختن یا خوانده نشدن دکمه‌ها در شبیه‌سازی می‌شود.
- آستانه رفع لرزش کلید (Debounce) و ریست ۱۵۰ میلی‌ثانیه‌ای: کلیدهای فیزیکی هنگام فشرده شدن دچار نوسانات ریز ولتاژ (Bounce) می‌شوند که باعث می‌شود میکروکنترلر یک بار فشرده شدن کلید را چندین بار ثبت کند (مثلاً به جای عدد ۲، عدد ۲۲۲ تایپ شود). برای حل این مشکل، یک سیستم شمارنده هوشمند در وقفه قرار دادیم. سیستم باید ۱۵ چرخه متوالی (معادل ۱۵۰ میلی‌ثانیه) وضعیت «عدم فشرده شدن کلید» را رصد کند تا حافظه کلید قبلی (last_registered_key) پاک شود. این زمان‌بندی دقیق باعث می‌شود هر چقدر هم دکمه را نگه دارید، عدد فقط یک بار تایپ شود.
- تخلیه اتمی حافظه (Atomic Data Flushing): برای حل مشکل نمایش پیام خطا در زمان جابجایی بین منوها، دستور memset(input_buffer, 0, NUMBER_OF_KEYS); را در ابتدای ورود به هر منو قرار دادیم. این دستور به صورت یکپارچه (Atomic) کل آرایه حافظه پسورد را قبل از اینکه کاربر شروع به تایپ کند، کاملاً صفر و پاک‌سازی می‌کند تا کاراکترهای شانسی یا باقیمانده از منوهای قبلی، محاسبات رمز عبور را خراب نکنند.

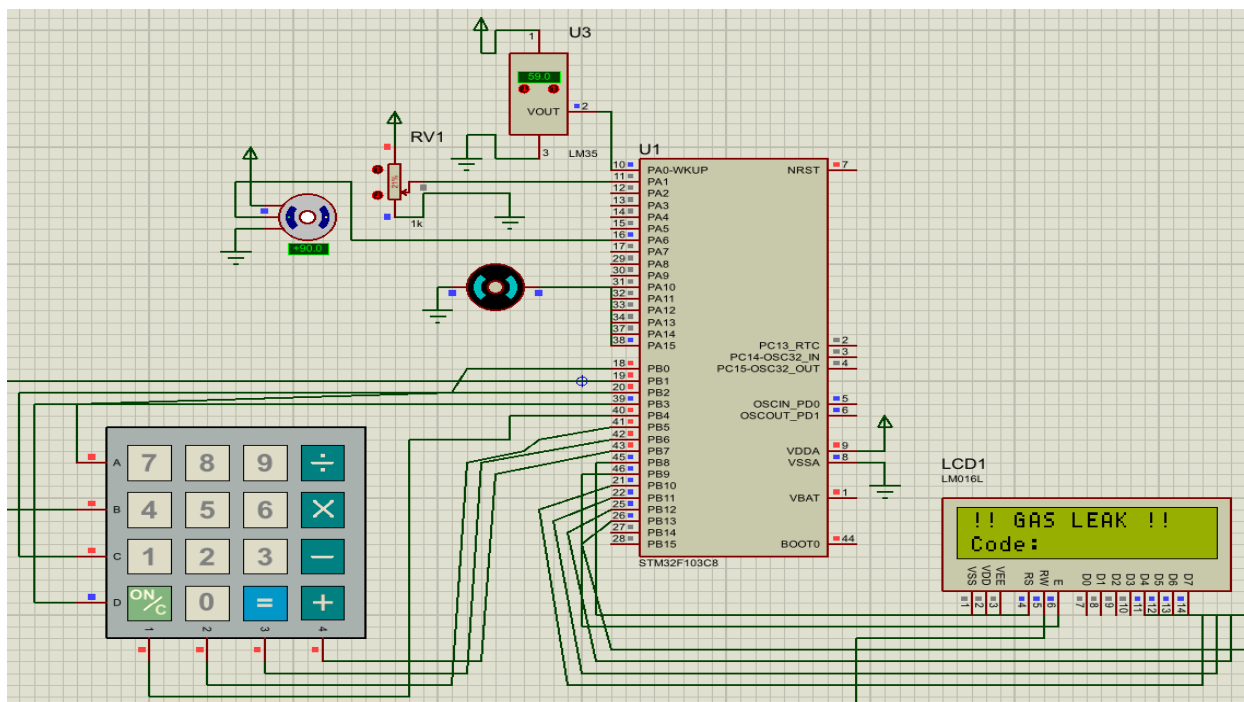
5. نتایج حاصل از پیاده‌سازی و تست سیستم

سیستم نهایی به صورت جامع در محیط شبیه‌ساز پروتئوس (Proteus) مورد پیاده‌سازی و کالبراسیون قرار گرفت و نتایج تجربی زیر به صورت دقیق به دست آمد:

۱. واکنش بلادرنگ و سریع به حرارت: با افزایش دمای محیط و رسیدن مقادیر سنسور LM35 (کانال ۰ مبدل ADC1) به عدد دیجیتال 491 (معادل آستانه خطر 60 درجه سانتی‌گراد)، پمپ آب پاش حریق (DC Motor) از طریق سیگنال PWM تایمر ۲ بلافاصله با حداکثر سرعت (دیوتی سیکل ۱۰۰٪) فعال گردید.
۲. واکنش هماهنگ و دقیق به نشت گاز: با چرخاندن پتانسیومتر شبیه‌ساز گاز (کانال ۱ مبدل ADC1) و عبور از آستانه دیجیتال ۱۵۰۰، سیستم در کمتر از چند میلی‌ثانیه خطر را ردیابی کرد. در این لحظه، سیگنال تایمر ۳ پالس خود را از ۱۰۰ به ۲۰۰ تغییر داد و سروو موتور شیر قطع‌کن فیزیکی گاز را دقیقاً به زاویه ۹۰ درجه (حالت کاملاً مسدود) چرخاند.
۳. پایداری ضریب امنیت هویت و منوی تعاملی: سیستم در برابر حملات تزریق کد یا فشردن مکرر کلیدهای اشتباه، مقاومت ۱۰۰٪ از خود نشان داد. منوی تعاملی LCD با ماسک کردن کاراکترهای رمز به صورت *، امنیت حریم خصوصی اپراتور را حفظ کرده و وضعیت‌های امنیتی یا بحرانی سیستم را به صورت بلادرنگ گزارش داد.



شکل 5.1: عکس از محیط پروتئوس در لحظه فعال شدن پمپ آب و نمایش !!FIRE ALARM!! روی LCD



شکل 5.2: عکس از محیط پروتئوس در لحظه چرخیدن سروو موتور به زاویه 90 درجه و نمایش !!GAS LEAK!! روی LCD

6. چالش‌های فنی، عیب‌یابی (Debug) و نحوه حل آن‌ها

در طول فرآیند توسعه این پروژه سخت‌افزاری، با چالش‌های فنی متعددی در سطح رجیسترهای داخلی میکروکنترلر و فیزیک شبیه‌ساز مواجه شدیم که به صورت گام‌به‌گام و بر اساس اصول مهندسی تعبیه‌شده حل شدند:

۱. خفگی پردازنده در پورت سریال (UART) و قفل شدن دریافت رمز

- **تشریح مشکل:** در فازهای اولیه پروژه که از ترمینال کامپیوتر (UART) استفاده می‌کردیم، دستورات سنگین چاپ متن (HAL_UART_Transmit) را درون تابع وقفه گیرنده (Rx-Callback) قرار داده بودیم. این ساختار اشتباه باعث می‌شد پردازنده تمام پهنای باند خود را صرف ارسال کاراکترهای متنی کند و کاراکترهای بعدی رمز عبور که از کیبورد می‌آمدند را از دست بدهد (Data Loss) و سیستم قفل کند.
- **نحوه حل:** تابع وقفه سریال را به سبک‌وزن‌ترین حالت ممکن (Lightweight) تبدیل کردیم؛ به طوری که در وقفه سخت‌افزاری تنها یک پرچم (Flag) فعال شده و کاراکتر ذخیره می‌شد. سپس تمام پردازش‌های سنگین مربوط به نمایش متن و تحلیل رمز عبور را به حلقه اصلی (while) منتقل کردیم تا بستر سیستم کاملاً غیرمسدودکننده (Non-blocking) شود.

۱۱. عدم پاسخگویی و قفل شدن سروو موتور روی زاویه صفر

- تشریح مشکل: سروو موتور به فرکانس دقیق ۵۰ هرتز نیاز داشت. با وجود تنظیم دقیق ریاضی Prescaler روی ۷۹ و ARR روی ۱۹۹۹ در فرکانس کلاک ۸ مگاهرتز، موتور در محیط شبیه‌ساز هیچ واکنشی به مقادیر جدید رجیستر CCR نشان نمی‌داد و روی صفر درجه قفل بود.

- نحوه حل: پس از بررسی عمیق دیتاشیت مشخص شد که مقادیر سایه (Shadow Registers) در تایمرهای STM32 تا زمان رخ دادن یک رخداد به‌روزرسانی (Update Event) اعمال نمی‌شوند. با اضافه کردن دستور اجباری و سخت‌افزاری `htim3.Instance->EGR = TIM_EGR_UG`; کلاک تایمر را وادار به بارگذاری فوری مقادیر کردیم و با اصلاح اتصالات تغذیه موتور در پروتئوس، سروو به کار افتاد.

III. خطا در محاسبات ریاضی کالیبراسیون سنسور دما

- تشریح مشکل: در ابتدا عدد آستانه خطر آتش‌سوزی را بدون محاسبات و بر اساس حدس سخت‌افزاری روی ۱۵۰۰ تنظیم کرده بودیم. اما در زمان تست، متوجه شدیم سیستم هرگز وارد فاز حریق نمی‌شود. پس از فرمول‌نویسی متوجه شدیم عدد ۱۵۰۰ در یک میکروکنترلر ۱۲ بیتی با ولتاژ مرجع ۳.۳ ولت، معادل دمای غیرواقعی بالای ۱۲۰ درجه سانتی‌گراد است!
- روش حل: با اعمال محاسبات دقیق رزولوشن مبدل (4096 گام) و انطباق آن با خروجی سنسور LM35 که به ازای هر درجه سانتی‌گراد 10 mV تولید می‌کند، رابطه ریاضی را اصلاح کردیم. عدد آستانه را روی مقیاس 491 کالیبره کردیم تا سیستم دقیقاً و با پایداری کامل روی دمای استاندارد 60 درجه سانتی‌گراد واکنش نشان دهد.

IV. تداخل رم‌ها هنگام تغییر منو

- تشریح مشکل: جابجایی بین منوها باعث به هم ریختن آرایه رمز می‌شد.
- روش حل: همان‌طور که در بخش زمان‌بندی اشاره شد، از تکنیک تخلیه اتمی حافظه با دستور `memset` استفاده کردیم تا بافر پیش از هر ورودی جدید پاکسازی شود.

V. قفل شدن زاویه سروو موتور در 0.18 درجه

- تشریح مشکل: به دلیل خطای گرد کردن (Rounding) در شبیه‌ساز پروتئوس، پالس PWM به صورت زاویه کاملاً صفر دیده نمی‌شد.
- روش حل: تنظیمات Minimum Angle خود قطعه در پروتئوس را روی 0.0 تنظیم کردیم و در کد مقدار مقایسه را روی 100 (معادل ۱ میلی‌ثانیه) قرار دادیم تا کاملاً کالیبره شود.

VI. نویز شدید، تایپ خودکار کاراکترهای و فرار فیزیکی کپی

- تشریح مشکل: پس از حذف پورت سریال و اضافه کردن کیپد ۴x۴ و LCD موازی ۶ پینی به پورت B، سیستم دچار "هذیان الکتریکی" شد. بدون اینکه کلیدی لمس شود، کاراکترهای روی LCD نقش می‌بستند و ستون‌های کیپد در پروتئوس به رنگ خاکستری (حالت شناور یا Floating) در می‌آمدند. همچنین فرکانس بالای میکرو اجازه ثبت کلیدها را به فیزیک شبیه‌ساز نمی‌داد.
- روش حل: این مشکل دو ریشه داشت. اول وضعیت پایه‌های ستون بود که با فعال‌سازی نرم‌افزاری مقاومت‌های بالاکش داخلی میکروکنترلر (GPIO_PULLUP) روی پین‌ها ولتاژ آن‌ها را به ۵ ولت ثابت فیکس کردیم. دوم، بازطراحی الگوریتم اسکن تایمر ۴ بود؛ به طوری که به جای اسکن کل سطرها در چند میکروثانیه، ساختار را به صورت "تک‌سطر در هر ۱۰ میلی‌ثانیه" تغییر دادیم تا موتور شبیه‌سازی پروتئوس فرصت فیزیکی برای پردازش افت ولتاژ کلید را داشته باشد.

7. مواردی که مدنظر بود اما اجرا نشد (محدودیت‌ها و توسعه آینده)

در طول پیشبرد پروژه ایده‌های پیشرفته‌ای مدنظر گروه بود که به دلایل فنی و محدودیت‌های نرم‌افزاری شبیه‌ساز، پیاده‌سازی آن‌ها به فازهای توسعه آینده موکول شد:

I. **عدم موفقیت در راه‌اندازی نمایشگر به صورت I2C:** هدف اولیه ما کاهش سیم‌کشی‌های LCD از ۶ پین به ۲ پین با استفاده از ماژول درایور PCF8574 تحت پروتکل I2C بود. اما توابع درایور سخت‌افزاری (HAL_I2C_Master_Transmit) به دلیل عدم پشتیبانی کامل شبیه‌ساز پروتئوس از خاصیت کشیدگی کلاک (Clock Stretching) و تایمینگ‌های رجیستری STM32، در یک حلقه بی‌پایان وضعیت HAL_BUSY گیر می‌کردند و میکروکنترلر فریز می‌شد. در نتیجه برای حفظ پایداری، از پروتکل موازی استاندارد استفاده شد.

II. **عدم استفاده از قابلیت سگ نگهبان آنالوگ (Analog Watchdog):** در نظر داشتیم به جای بررسی مداوم مقادیر سنسورها در حلقه اصلی while(1)، از قابلیت سخت‌افزاری سگ نگهبان آنالوگ در واحد ADC استفاده کنیم تا خود سخت‌افزار در صورت عبور ولتاژ از حد مجاز، مستقیماً وقفه تولید کند. با توجه به ناپایداری شبیه‌ساز در مدیریت کلاک این واحد، پیاده‌سازی نرم‌افزاری در حلقه اصلی به عنوان یک راه‌حل جایگزین و پایدارتر انتخاب گردید.

III. **عدم پیاده‌سازی سیستم‌عامل بلادرنگ (FreeRTOS):** قصد داشتیم سیستم را بر بستر تسک‌های زمان‌بندی‌شده سیستم‌عامل سوار کنیم. اما با توجه به ماهیت پروژه و برای جلوگیری از سربار (Overhead) غیرضروری پردازنده، معماری ماشین حالت Bare-Metal طراحی شد که با سرعت میکروثانیه به خوبی پاسخگوی نیاز سیستم بود.

8. نتیجه‌گیری نهایی

این پروژه اثبات کرد که تفاوت بسیار زیادی بین کدنویسی ساده و «مهندسی سیستم‌های تعبیه‌شده» وجود دارد. ما موفق شدیم با بهره‌گیری از توانمندی‌های سخت‌افزاری پردازنده‌های ARM Cortex-M3 (نظیر تایمرهای پیشرفته، وقفه‌ها و مقاومت‌های داخلی)، سیستمی را خلق کنیم که به جای پردازش خطی و کند، به صورت موازی (Parallel) می‌اندیشد.

چالش‌هایی که در مسیر همگام‌سازی کدهای C با موتور شبیه‌ساز پروتئوس داشتیم، دیدگاه ما را نسبت به زمان‌بندی (Timing) در ابعاد میلی‌ثانیه و میکروثانیه، مدیریت حافظه RAM و مدیریت جریان برنامه بسیار عمیق‌تر کرد. سیستم نهایی نه تنها یک هشداردهنده حریق، بلکه یک پنل صنعتی HMI کامل با قابلیت‌های امنیتی، کنترل دستی و واکنش بلادرنگ به خطرات است.

میزان مشارکت اعضا در پروژه

۱. نوید رستمی

- طراحی ماشین‌های حالت سیستم و یکپارچه‌سازی کل پروژه
- طراحی و راه اندازی نمایشگر و کیپد (توابع منطقی و امنیتی + اسکن)
- طراحی پروتئوس پروژه

۱۱. معید غیادی

- طراحی سنسور دما و پمپ آب

۱۱۱. امیرحسین اصغری

- طراحی سنسور گاز و شیر گاز
- طراحی پاورپوینت پروژه

۱۱۱۱. علی مددی

- طراحی سنسور گاز و شیر گاز

۱۱۱۱۱. پوپک موسوی

- طراحی سنسور دما و پمپ آب